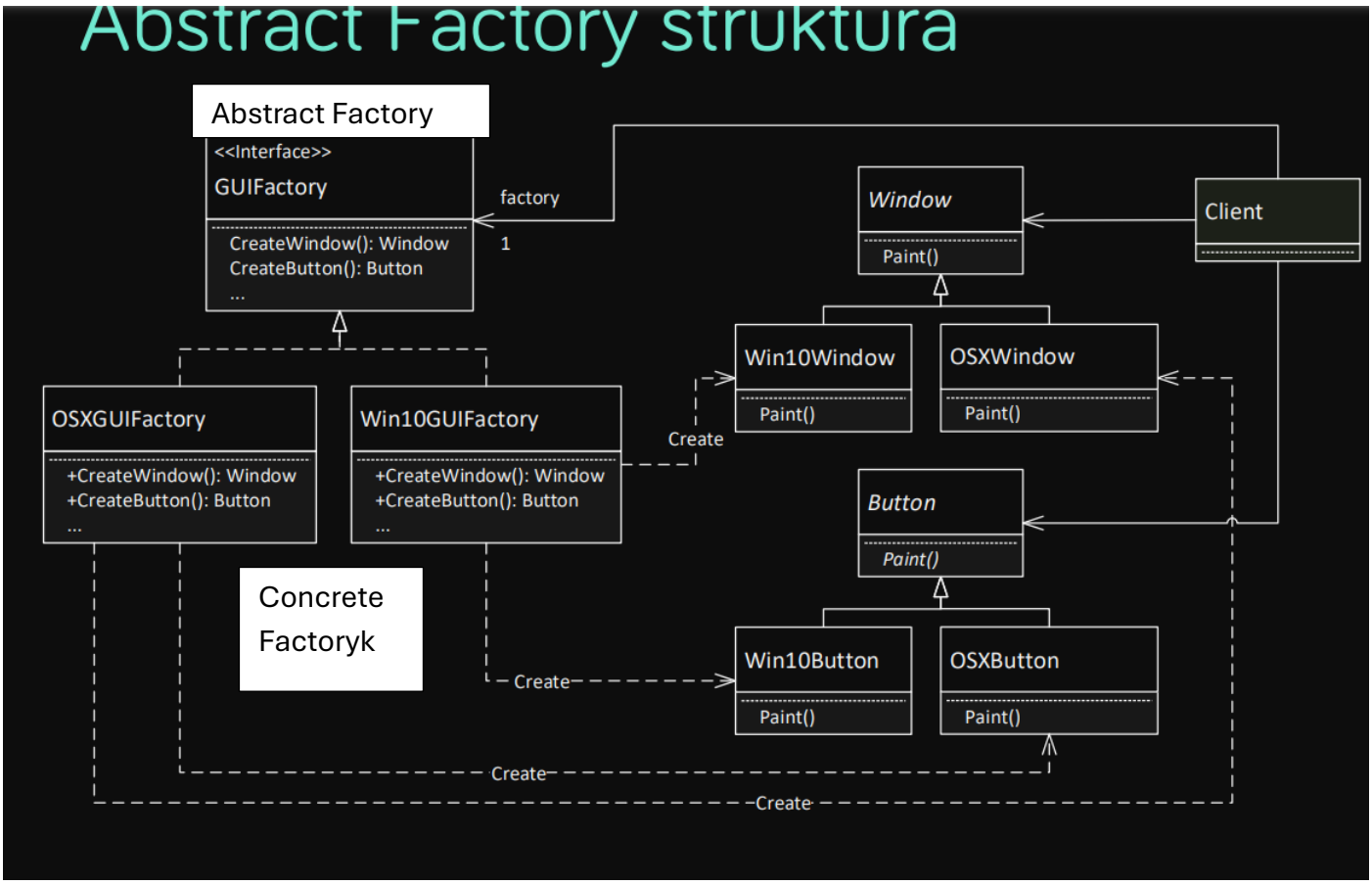
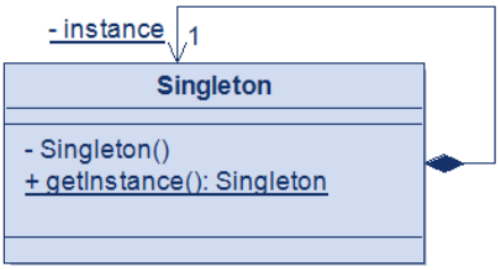
Példa egyszerűen:

- **Abstract Factory:** Egy absztrakt osztály vagy interfész, ami azt mondja: „Gyártsd le a széket, asztalt és kanapét."
- **Concrete Factory:** Ez dönti el, hogy modern vagy klasszikus bútorkészletet készítsen.
- **Termékosztályok:** A konkrét szék, asztal és kanapé osztályai (ModernSzék, ModernAsztal, KlasszikusSzék, stb.).



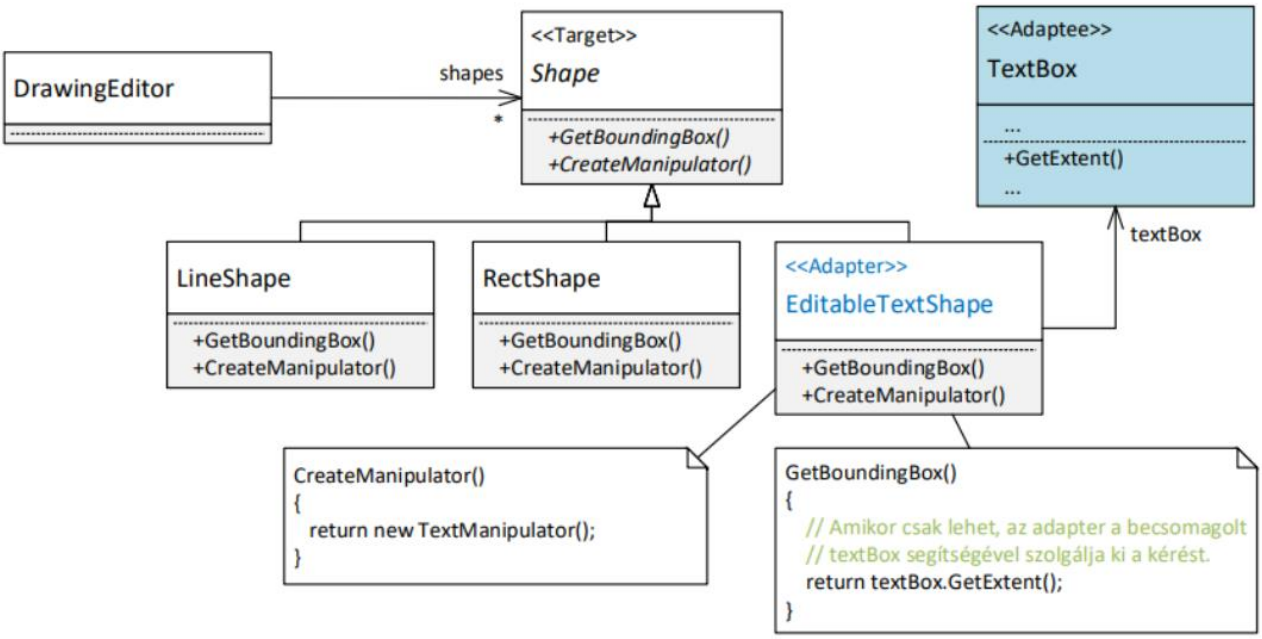
- Bárhonnan könnyen elérhető globális eszköz.
- Egy statikus változóban tárolja önmagában az egyetlen példányát. Védett konstruktort használ, hogy ne tudjanak többet létrehozni belőle. Van egy statikus példány lekérő funkciója. (ha nem statikus a funkció, lekérhetetlen, mert létrehozni sem lehet)

Általános mintázat:

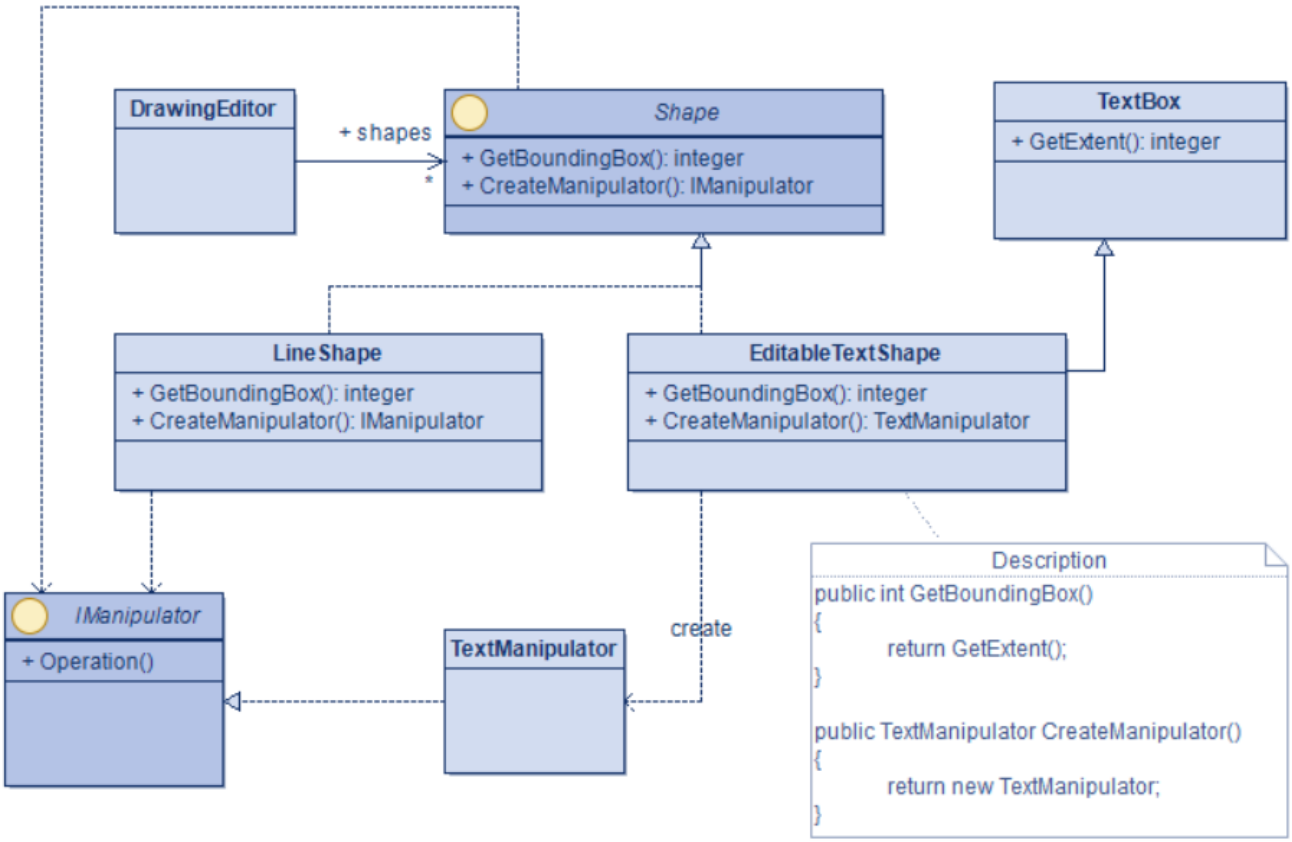



```
class WindowManager
{
    // Tárolja az egyetlen példányt, kezdetben null.
    private static WindowManager instance;

    // Glogális hozzáférést biztosító statikus művelet
    public static WindowManager GetInstance()
    {
        // Az egy példányt az első hozzáférés alkalmával hozzuk létre
        if (instance == null)
            instance = new WindowManager();
        return instance;
    }
    // Védett konstruktor
    protected WindowManager() { }

    // Az osztály egyik művelete
    public void DoSomething() { /*...*/}
}

…
// Valahol a kódban hozzáférünk hozzá a WindowManager singleton példányhoz
WindowManager.GetInstance().DoSomething();
```

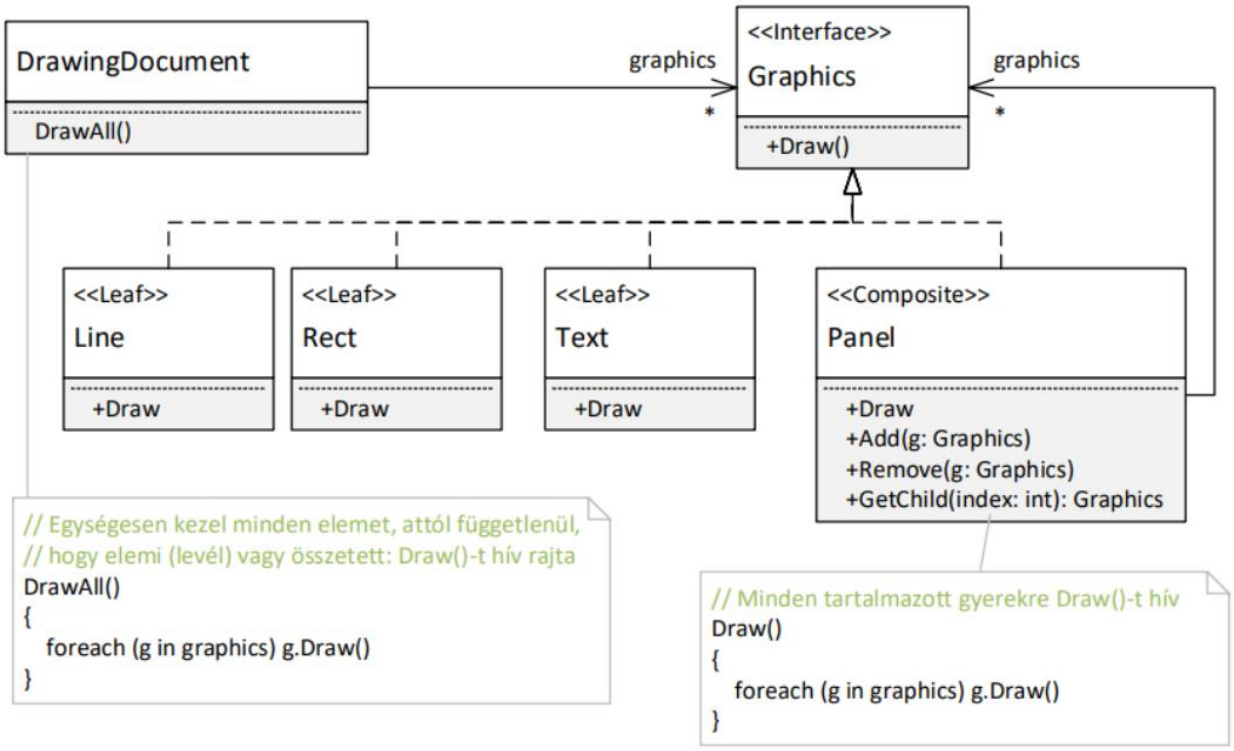
Object Adapter : Adapter és Adaptee között asszociáció nem öröklés! TextBox textBox = new...

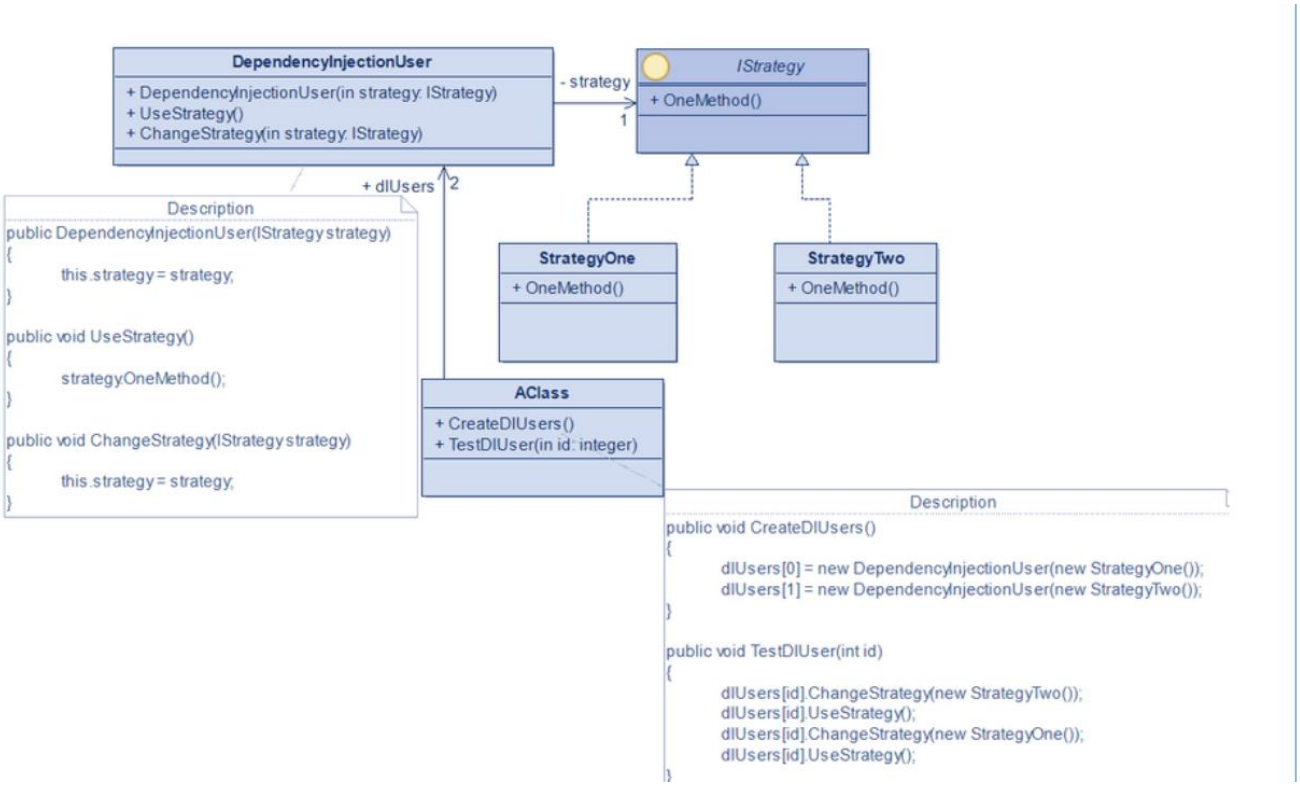


Class Adapter : Adapter és Adaptee között öröklés!

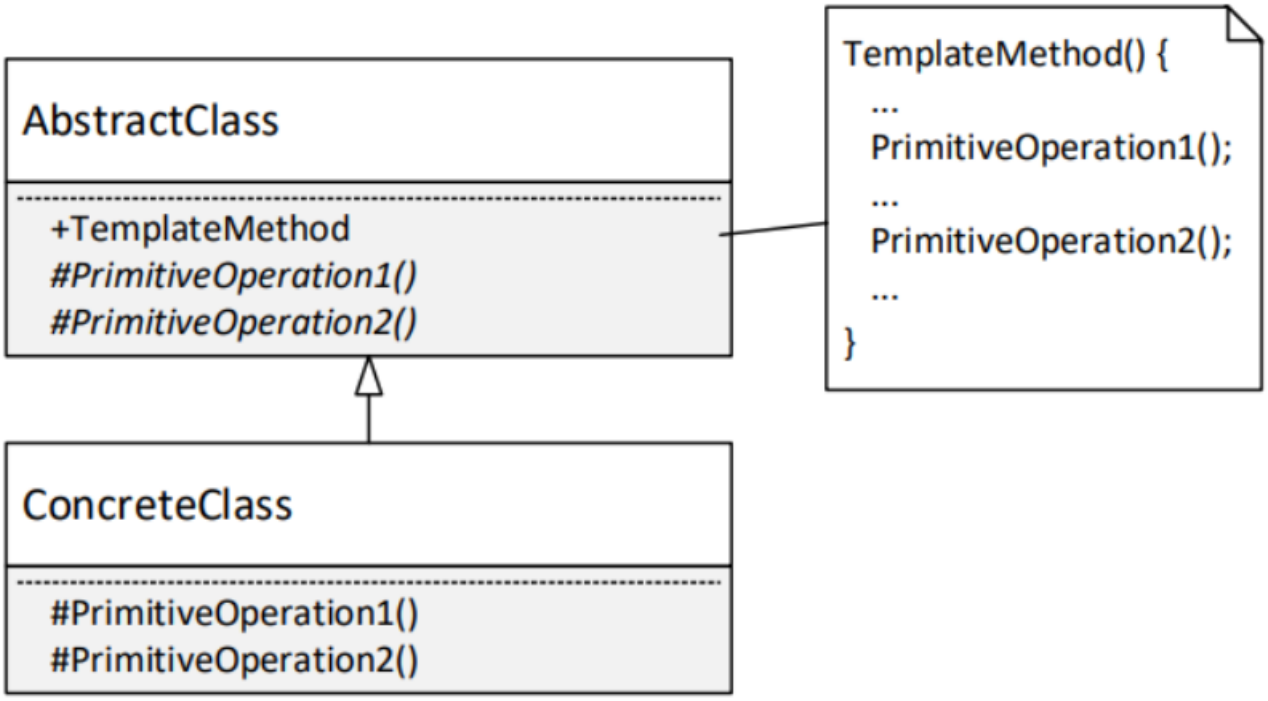


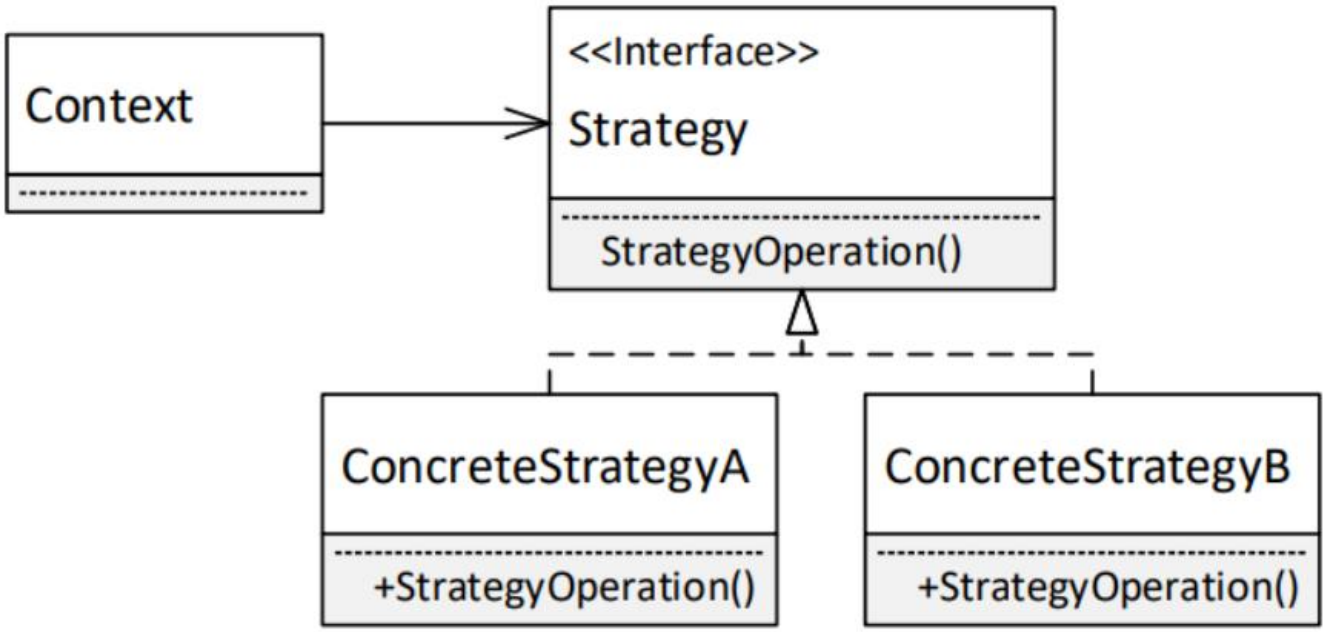
Composite példa

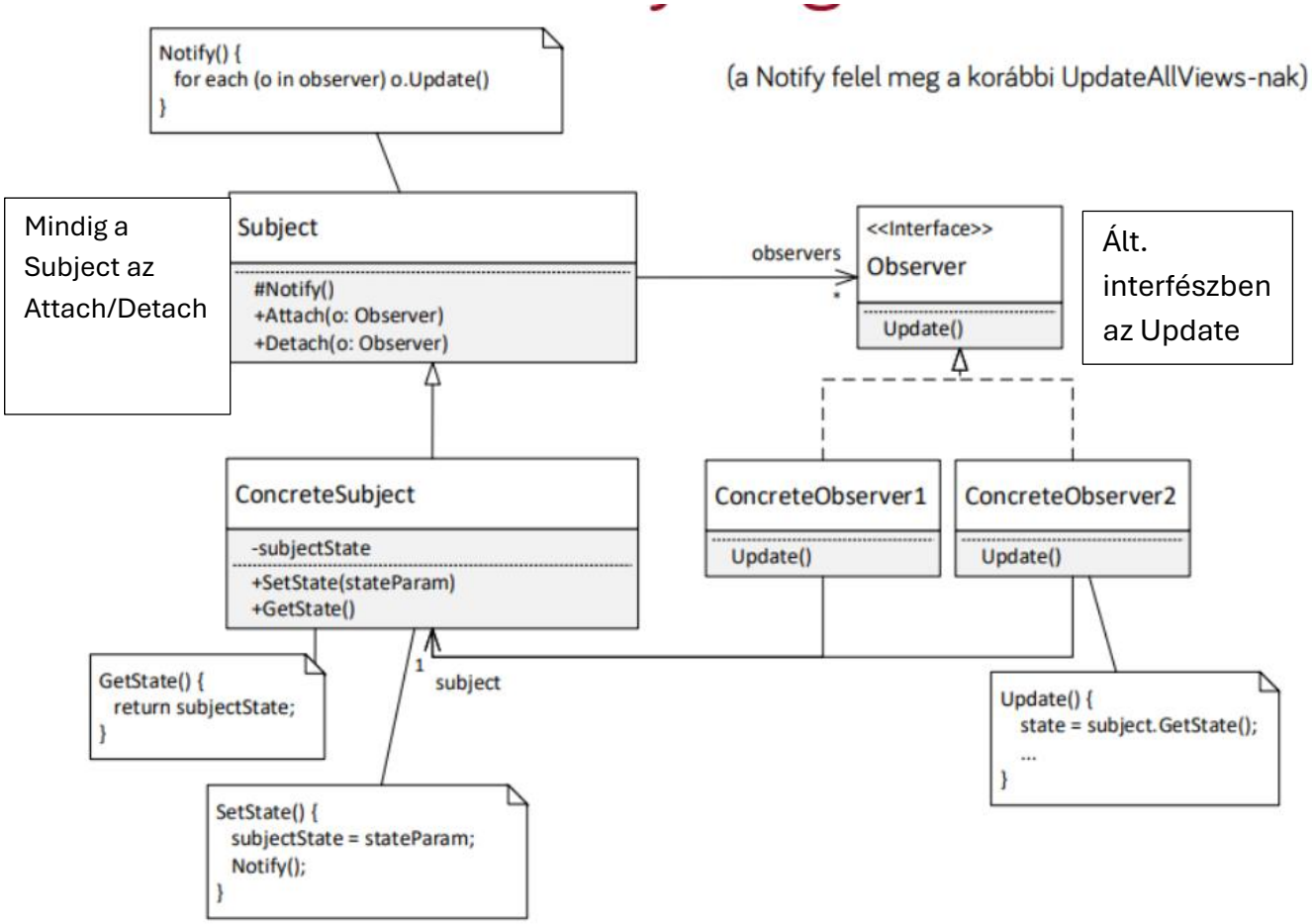


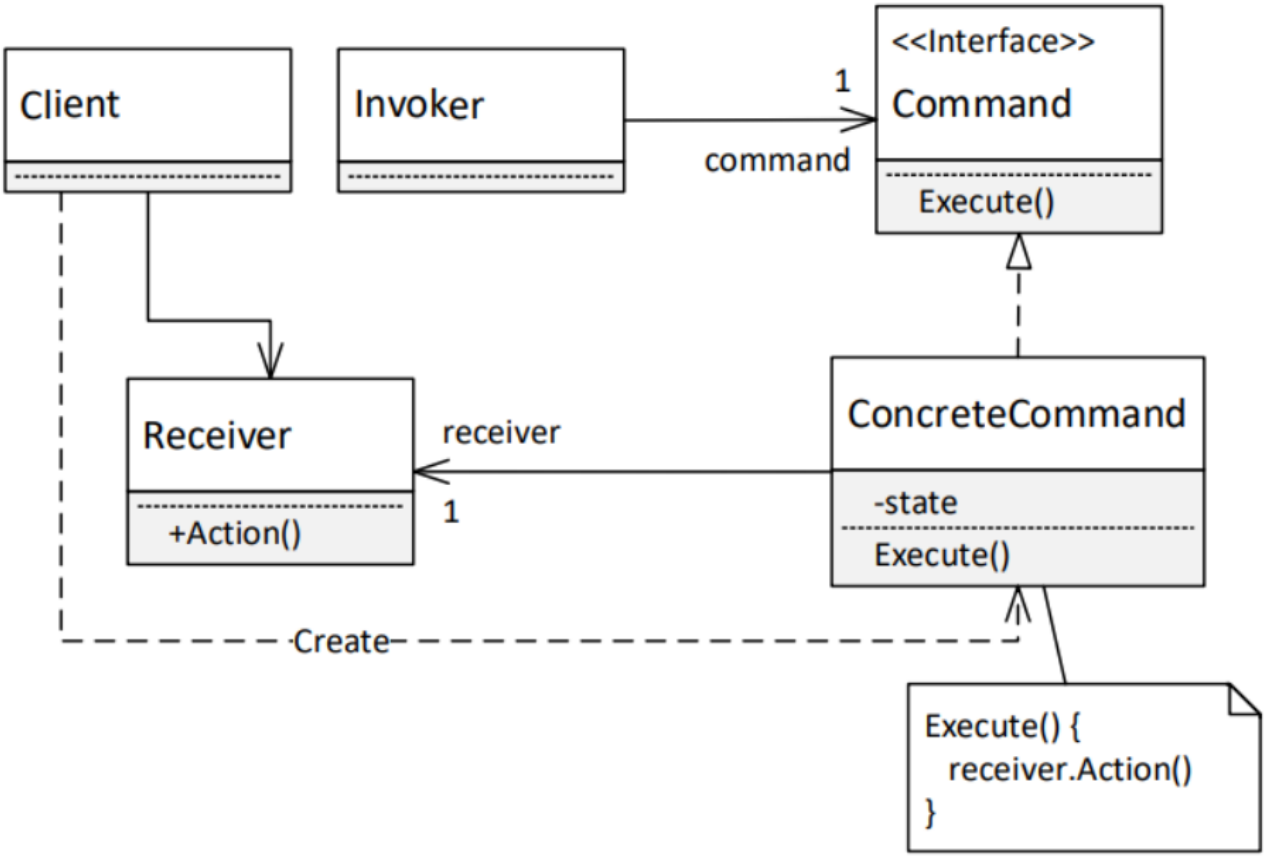


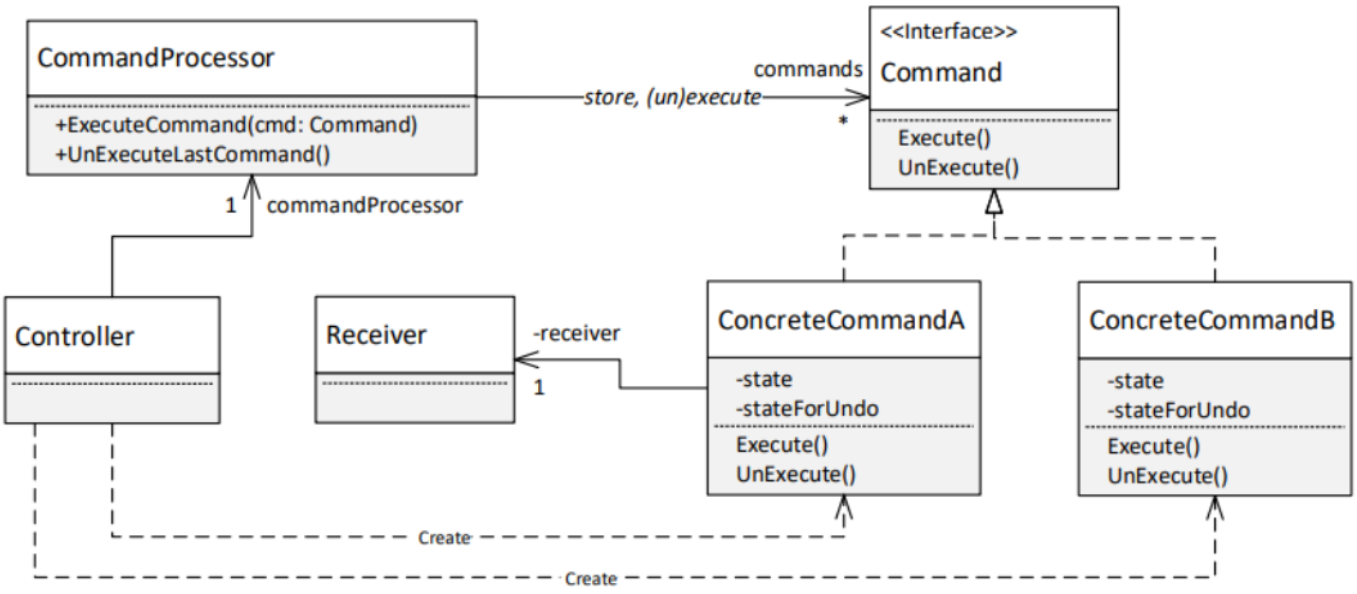

```
class SecurityService
{
    // Interfészként hivatkozunk a user repository implementációra
    IUserRepository userRepository;

    // Konstruktor paraméterként kap egy  IUserRepository implementációt
    public SecurityService(IUserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // A userId azonosítójú felhasználó jelszavát megváltoztatja
    // newPassword-re (ha teljesíti a validációs szabályokat)
    // Visszatérési értékben jelzi, hogy sikerült-e a jelszó megfelel-e
    // a validálási szabályoknak.
    public bool ChangePassword(int userId, string newPassword)
    {
```

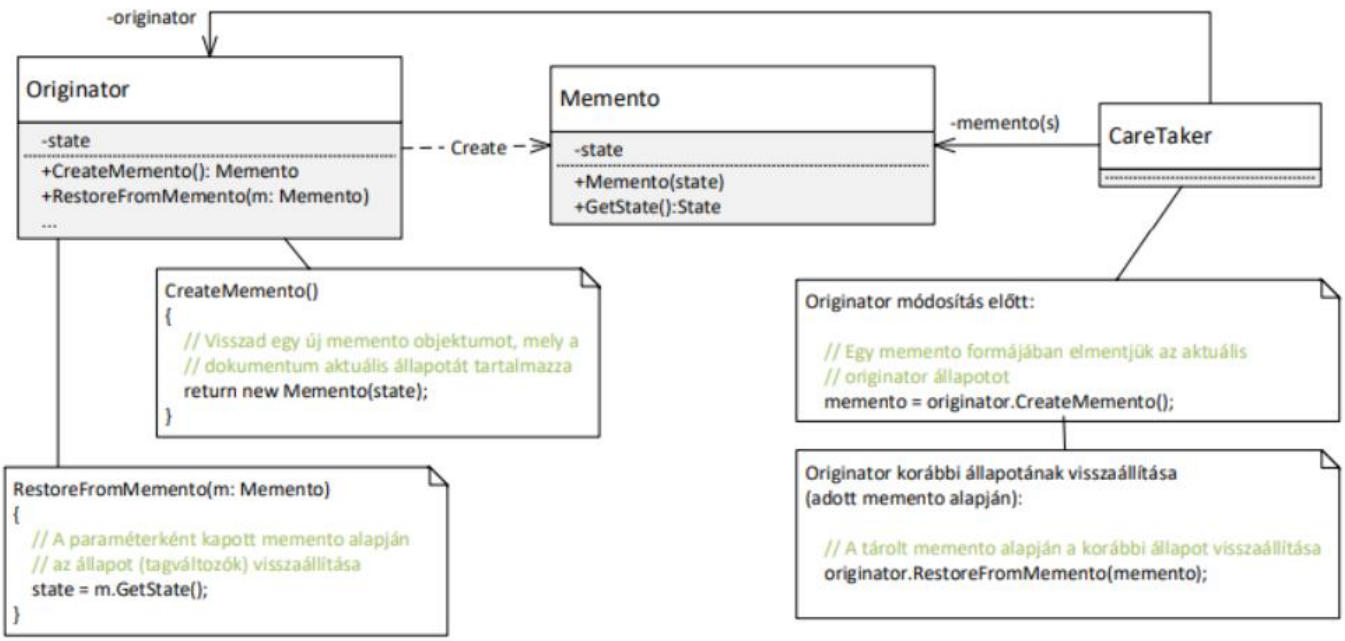

AbstractClass
+TemplateMethod
#PrimitiveOperation1()
#PrimitiveOperation2()
TemplateMethod() {
...
PrimitiveOperation1();
...
PrimitiveOperation2();
...
}
ConcreteClass
#PrimitiveOperation1()
#PrimitiveOperation2()


Context
<<Interface>>
Strategy
StrategyOperation()
ConcreteStrategyA
+StrategyOperation()
ConcreteStrategyB
+StrategyOperation()


Notify() {
for each (o in observer) o.Update()
}
(a Notify felel meg a korábbi UpdateAllViews-nak)
Mindig a Subject az Attach/Detach
Subject
#Notify()
+Attach(o: Observer)
+Detach(o: Observer)
observers
*
<<Interface>>
Observer
Update()
Ált. interfészben az Update
ConcreteSubject
-subjectState
+SetState(stateParam)
+GetState()
ConcreteObserver1
Update()
ConcreteObserver2
Update()
1
subject
GetState() {
return subjectState;
}
SetState() {
subjectState = stateParam;
Notify();
}
Update() {
state = subject.GetState();
...
}


Client
Invoker
1
command
<<Interface>>
Command
Execute()
Receiver
+Action()
receiver
1
ConcreteCommand
-state
Execute()
Create
Execute() {
receiver.Action()
}


CommandProcessor
+ExecuteCommand(cmd: Command)
+UnExecuteLastCommand()
store, (un)execute
commands
*
<<Interface>>
Command
Execute()
UnExecute()
1
commandProcessor
Controller
Receiver
-receiver
1
ConcreteCommandA
-state
-stateForUndo
Execute()
UnExecute()
ConcreteCommandB
-state
-stateForUndo
Execute()
UnExecute()
Create
Create



Mementó minta működése:

1. **Egy objektum állapotának elmentése**:
   - Az **Originator** létrehoz egy **Memento** objektumot, amely tárolja az állapotot.
   - Az **Originator** tárolja a **Memento** objektumot.
2. **Állapot visszaállítása**:
   - Ha szükséges, a **Caretaker** kérheti a **Memento** objektumot az **Originator**-tól, amely tartalmazza a mentett állapotot.
   - Az **Originator** a **Memento** segítségével visszaállítja az állapotot.

Originator: Ő neki az állapotát kell elmenteni, ő tartalmazza a belső állapotot. Ő is állítja vissza az állapotot.

Memento: Ő menti el az **Originator** állapotát. CSAK állapotot tárol!

CareTaker: A **Caretaker** lehetőséget biztosít arra, hogy az objektum állapotát vissza lehessen állítani anélkül, hogy a **Memento** belső működéséhez hozzáférnénk. **Memento** objektumok tárolásáért felelős. (A **Caretaker** visszaadja a **Memento**-t az **Originator**-nak.)